Recombination Does Not Help: a Kernel-Gated Measurement of Crossover in Program Synthesis, and the Verifier Artifact That Made It Affordable

Xavier Callens

SocrateAI Scientific Research & Formal Verification Laboratory

Edition 2, 24 October 2026. **Not peer reviewed.**

Abstract

Crossover is the operator that distinguishes a genetic algorithm from a hill-climber, and in evolutionary program synthesis its contribution is usually asserted rather than measured. Three preceding papers in this programme asked whether recombination is *detectable* in evolved programs and refuted the question's instrument each time. None asked the prior question: does recombination *help*?

That question needs a formal gate over hundreds of candidates, and the gate cost one proof-assistant invocation per candidate, which put a properly powered paired experiment out of reach. This paper reports, first, that the cost was an artifact of our own verifier wrapper rather than of the proof assistant. A Lean 4 compile exits nonzero on a batch containing a false obligation but still prints a per-declaration axiom line for every candidate, the failing ones carrying `sorryAx`. Our wrapper discarded those lines the instant the exit code was nonzero, which made a batch appear atomic and motivated three successive rounds of isolation machinery — a bisection, a flat fall-back measured as better than the bisection, and a break-even law $(1-p)^b > 1/b$ derived from the cost of isolating a failure. All three were solving a problem the wrapper had invented. Reading the kernel's own report gives every verdict in one invocation at any failure rate, measured at 1 invocation for $0, 1, 2, 4$ and 8 failures of 8, with every culprit named and no false candidate accepted.

With the gate affordable, we measure crossover directly. Fitness is exact rational agreement with a specification at sample points, computed outside the proof assistant; acceptance is the Lean kernel, so a wrong fitness can misguide the search but cannot manufacture an acceptance. Over 200 paired seeds on a calibrated target, mutation-only solved 94/200 (47.0%) against crossover's 58/200 (29.0%), Fisher exact $p = 2.98 \times 10^{-4}$. Conditional on solving, the evaluation count is unchanged (median ratio 0.959 over 35 pairs, sign test $p = 1.000$). Crossover therefore does not make successful searches faster; it makes success less likely — the signature of a macromutation destroying assembled structure rather than a recombination transmitting it.

We give the mechanism exactly. An expression is affine in any one subtree's value, $\text{out} = A + Bv$, and B is computable in exact rationals by one forward and one backward pass. A donor contributing $B_{\text{donor}}v$ in its own parent contributes $B_{\text{site}}v$ after transplant, so its contribution is rescaled or sign-inverted whenever the coefficients differ, even though its syntax arrives intact. We then test the repair this diagnosis suggests — restricting recombination to *homologous* sites, those whose coefficient sign pattern matches the donor's, by analogy with homologous recombination in biology. **The repair fails, and its own negative control refutes the mechanism.** Homology-restricted recombination reaches 70/200 (35.0%), neither significantly above random crossover ($p = 0.238$) nor recovering mutation-only, which beats it significantly ($p = 0.019$); and the anti-homologous control, restricted to *inverted* sign patterns and predicted to be worse, instead scores higher at 79/200 (39.5%) and is statistically indistinguishable from the treatment ($p = 0.408$) — the outcome

we had registered in advance as meaning the signature carries nothing. A fallback-to-mutation confound is excluded quantitatively. **Edition 2 tests the successor edition 1 recorded as untested** — inserting the donor *rescaled* so its contribution transfers exactly, verified exact at 1614 of 1614 comparisons — and its pre-registered falsifier fires too: 73/200 (36.5%), not significantly above random crossover ($p = 0.136$), still significantly below mutation-only ($p = 0.042$), and indistinguishable from the weaker sign-matching operator ($p = 0.835$). Six independent results now agree that subtree recombination in this representation does not help, including both repairs its exact diagnosis recommends.

Every numeric claim is reproduced by a script in the accompanying repository, and every reported defect of our own apparatus is recorded with the measurement that exposed it.

Contents

1 Introduction **2**
- 1.1 What this paper reports . . . 3
- 1.2 What this paper does not claim . . . 3

2 The atomicity artifact **3**
- 2.1 What we believed, and recorded as measured . . . 3
- 2.2 What is actually the case . . . 4
- 2.3 Why reading past the exit code is still sound . . . 4
- 2.4 The cost of the belief . . . 4

3 Measuring crossover **5**
- 3.1 Design . . . 5
- 3.2 Calibration, and a non-discriminating first attempt . . . 5
- 3.3 Controls . . . 5
- 3.4 Result . . . 6

4 The mechanism **6**

5 Homologous recombination, and why the repair fails **6**
- 5.1 The prediction, registered before the run . . . 7
- 5.2 Result . . . 7
- 5.3 Ruling out the obvious confound . . . 7
- 5.4 What survives . . . 7

6 Preserving the contribution exactly **8**
- 6.1 The operator . . . 8
- 6.2 A sharper control than edition 1's . . . 8
- 6.3 Result . . . 8
- 6.4 The falsifier fired . . . 9
- 6.5 What the data do support, at their real strength . . . 9
- 6.6 What this leaves . . . 9

7 Reproduction **9**

8 Related work and honest placement **9**

1 Introduction

A genetic algorithm differs from a hill-climber in exactly one place: it recombines. Everything else it does — mutate, evaluate, select — a mutation-only search also does. The value of crossover

is therefore the load bearing claim of the whole family, and in evolutionary program synthesis it is usually inherited from the schema-processing arguments of the string-genome era rather than measured on programs.

This programme has approached the question three times from the detection side. A quartet certificate refutes the four-point condition with an integer comparison a proof kernel can recheck, and a recombinant genealogy violates that condition where a clonal one does not; so if a program's behaviour were a heritable sequence, the certificate would report whether crossover had recombined. It is not. Behavioural profiles — a program's outcome at each of a battery of probes — failed the test on real evolved populations in every stratification we could construct, and a mutation-only arm, which is a tree by construction, carried certificates at the same rate as a crossover arm. An address-indexed ablation contribution, proposed as the heritable unit the profiles lacked, transmitted at a median fraction of 0.200–0.545 against a pre-registered bar of 0.70.

Each of those is a statement about *measurability*. None of them says whether crossover helps. That question requires running both arms to a verified solution often enough for the difference to be visible, and the verification is the expensive part.

1.1 What this paper reports

Two results, one methodological and one empirical, plus a mechanism and an attempted repair.

1. **The verification cost was ours, not Lean's** (§2). A compile containing a false obligation exits nonzero and still reports every declaration's axiom dependencies. Our wrapper threw that away on the exit code. Three rounds of isolation machinery were built on the resulting belief that a batch is atomic; none of it was needed.

2. **Crossover significantly reduces the solve rate** (§3). 47.0% against 29.0% over 200 paired seeds, $p = 2.98 \times 10^{-4}$, with the evaluation count unchanged conditional on success.

3. **The mechanism is the context coefficient** (§4). An expression is affine in a subtree's value and the coefficient depends on the operators above it, so a transplant preserves syntax and not contribution.

4. **The repair this suggests fails, refuted by its own control** (§5). Restricting recombination to sites with a matching coefficient sign pattern does not recover mutation-only performance, and the anti-homologous control is not worse than it.

1.2 What this paper does not claim

The measurement is of one search: a closed grammar $\{+, -, *\}$ over four variables and three rational constants, one calibrated target, one budget, one steady-state population scheme with tournament selection, one crossover rate. It is evidence about crossover in *this* setting and not a general theorem about recombination. The full specification target of the gate we use was solved by neither arm at this budget, so nothing here is established at the size the gate itself cares about. These limits are restated with the results rather than deferred to a closing paragraph.

2 The atomicity artifact

2.1 What we believed, and recorded as measured

Our verifier exposes a function that compiles a Lean file and returns one result per declared theorem, where a declaration is clean only if the source passes a forgery scan, the compile succeeds, and the kernel's `#print axioms` line for that declaration is free of `sorryAx` and of any axiom outside the whitelist {`propext`, `Classical.choice`, `Quot.sound`}.

Batching several candidates into one file amortises a large fixed cost: a candidate's own elaboration is about 0.14 s against about 5.23 s of fixed per-invocation overhead, so roughly 97% of a verification is paid whatever the file contains. But we observed that a batch containing one false obligation returned a single aggregate failure with no per-candidate verdict, and recorded that as a property of Lean: *a compile is atomic; one failing obligation fails the whole file and yields no per-declaration rows.*

2.2 What is actually the case

It is a property of the wrapper. The function returns an aggregate failure the instant the process exit code is nonzero, discarding output the kernel has already produced. A single invocation on eight candidates with two false exits 1 and prints all eight lines:

```
'At0.At0_m' depends on axioms:  [propext, Classical.choice, Quot.sound]
'At2.At2_m' depends on axioms:  [propext, sorryAx, Classical.choice,
Quot.sound]
```

A failed tactic proof is admitted as `sorryAx` and reported by name, which is exactly the condition the gate already rejects on.

2.3 Why reading past the exit code is still sound

Acceptance never rested on the exit code. It rests on three conditions that are unchanged: the source-forgery scan, which rejects `#exit` truncation and `#print` string spoofing; every declared theorem having its own axioms line, so a declaration that never elaborated prints nothing and fails as unchecked; and that line being free of `sorryAx` and of non-whitelisted axioms. A nonzero exit now says only that some declaration failed, which the per-name checks catch. Nothing is accepted that the stricter path would have accepted; the report is widened, not the trust surface.

2.4 The cost of the belief

Three rounds of machinery, each internally sound, all downstream of the wrong premise.

failures of 8	bisection	flat fall-back	individual	kernel report
0	1	1	8	**1**
1	7	9	8	**1**
2	11	9	8	**1**
4	15	9	8	**1**
8	–	9	8	**1**

Bisection was built first, on the assumption that isolating k failures in $O(k \log n)$ invocations beats re-running n. At 97% fixed cost every branch re-pays a near-whole verification, so it is worse than the flat re-run it was meant to improve on. The flat fall-back replaced it. Then a break-even law was derived — batching pays only while $(1-p)^b > 1/b$, giving a break-even failure rate near 23% at $b = 8$ — which made the high-failure regime an evolutionary search occupies look like a barrier to the very experiment this paper runs. With no isolation to charge for, the saving is b at any failure rate and the barrier does not exist.

In the shipped path, measured at 200 seeds of the experiment below, all distinct solutions were gated in one invocation with every culprit named and no false candidate accepted at any failure rate.

Remark 2.1. The general form is worth stating because it is cheap to guard against. A measured *limit* reported by code one owns is a property of that code until it is checked one layer below. The defect survived an existing guard — that a gate can be measuring one's own choice — because the wrapper's output looked like physics rather than like a choice.

3 Measuring crossover

3.1 Design

A genome is an expression tree over the binary operators $\{+, -, *\}$ with leaves drawn from the variables h, w, q, p and the constants $1, 2, \frac{1}{2}$. Fitness is the total absolute exact error against a target at twelve rational sample points, computed with arbitrary-precision rationals, so a fitness of zero means exact agreement at those points. Agreement is not equality; that is what the kernel is for.

The two layers are deliberately different. Fitness guides the search and costs no proof-assistant time. Acceptance is the Lean kernel, applied to the rendered candidate against the rendered target, proved by `ring` with its axiom line checked. A wrong fitness function would waste search effort and could never cause a wrong acceptance.

3.2 Calibration, and a non-discriminating first attempt

Our first attempt ran both arms against the full target and both solved 0/24, the best fitness reached anywhere being about 55.4 with nothing below 10. That is a non-discriminating comparison: two arms failing equally say nothing about recombination, and reporting it as evidence against crossover would have been reading a result off an instrument that cannot separate the arms.

Difficulty was therefore calibrated on a ladder of reachable targets of increasing size, with the rung chosen by a criterion fixed in advance: the *pooled* solve rate over both arms must lie in $[0.3, 0.7]$. Pooling is the point — selecting the rung at which crossover looked best would be choosing the apparatus to suit the answer.

rung	size	pooled rate	mutation	crossover
L1	5	0.750	13/16	11/16
L2	7	0.969	16/16	15/16
L3	9	**0.438**	9/16	5/16
L4	11	0.125	1/16	3/16
L5	13	0.000	0/16	0/16
L6	17	0.000	0/16	0/16

Only L3 fell in the band. Note that L2 is *easier* than L1 despite being larger: $q + hp - q$ reduces to hp, so semantic difficulty and node count come apart, which is itself a caution against reporting program-search difficulty by size.

3.3 Controls

The target is asserted reachable and scoring exactly zero before any arm runs, so a failure to solve is informative. An unreachable target — one requiring division by a variable, outside the grammar — was solved 0 times out of all control runs; an arm that solved it would indicate a fault in fitness or in the gate. Arms share seeds and initial populations.

3.4 Result

	solved	median evaluations	test
mutation-only	94/200 (47.0%)	1286.5	
crossover	58/200 (29.0%)	1269.0	Fisher $p = 2.98 \times 10^{-4}$
paired, both solved ($n = 35$)	ratio 0.959, cheaper on 18		sign test $p = 1.000$

The two measures disagree and that is the result. Crossover significantly reduces the probability of solving at all, while leaving the evaluation count of successful runs essentially untouched. It does not make successful searches faster; it makes success less likely.

Remark 3.1. At 48 seeds the paired median ratio was 1.500 over 10 pairs, which would have been reported as crossover being 50% more expensive. At 200 seeds and 35 pairs it is 0.959 with a sign test of $p = 1.000$. The small-sample paired estimate did not replicate; the solve-rate difference, only $p = 0.147$ at 48 seeds, is the measure that held. We record this because the first direction seen in a paired statistic on ten pairs is not a result.

4 The mechanism

Definition 4.1 (Context coefficient)**.** For an expression tree t over $\{+, -, *\}$ and an address a occurring once in t, the value of t is affine in the value v of the subtree at a: $t = A + Bv$, with A and B depending only on the rest of the tree and on the evaluation point. We call B the *context coefficient* of a.

B is exactly computable by reverse accumulation: one forward pass for every node's value, then one backward pass seeded with $B(\text{root}) = 1$, where $+$ sends B to both children, $-$ sends B left and $-B$ right, and $*$ sends $B \cdot \text{value(sibling)}$ to each child. This is $O(|t|)$ per sample point in exact rational arithmetic. We verified it against the defining two evaluations $B = t|_{v=1} - t|_{v=0}$ on random trees, with exact agreement over more than five hundred address–point pairs.

The diagnosis follows immediately. A donor subtree contributing $B_{\text{donor}}\, v$ inside its own parent contributes $B_{\text{site}}\, v$ after transplant. Unless the two coefficients agree, the donor's *contribution* is rescaled or sign-inverted even though its syntax arrived intact. Subtree crossover transmits structure and not function, which is what it means to call it a macromutation. This agrees with an independent finding on different evidence: transfer fidelity of a donor's ablation signature falls with the number of multiplicative operators on the root-to-insertion path.

Remark 4.1. The naive implementation of B — two whole-tree evaluations per address — costs $O(|t|^2)$ per point and measured at 5.5× the tree-point evaluations of a plain arm. Had the homology arms below been compared on fitness evaluations alone, that hidden cost would have made any win meaningless. Both cost metrics are reported for every arm.

5 Homologous recombination, and why the repair fails

Biology does not recombine at arbitrary positions; it recombines at homologous ones, and non-homologous rejoining is a damage pathway rather than a creative one. Random subtree crossover is the non-homologous analogue. The diagnosis of §4 suggests the corresponding repair: admit a donor only into a site that preserves its contribution, operationalised as requiring the sign pattern of B across the sample points to match the donor's.

5.1 The prediction, registered before the run

Recorded in the artifact before any arm executed: homology-restricted recombination should raise the solve rate above random crossover's 29.0%, ideally to at least mutation-only's 47.0%; anti-homologous recombination, restricted to *inverted* sign patterns, should do no better than random. We stated the falsifier explicitly — *if the two homology arms behave alike, the signature carries nothing and the mechanism is not what was claimed* — and counted fallbacks to mutation, taken when no admissible site exists, so that an arm quietly becoming mutation-only could not read as a success of recombination.

5.2 Result

arm	solved	rate	median tree-point evaluations
mutation-only	94/200	0.470	48 000
crossover (random)	58/200	0.290	48 000
homologous crossover	70/200	0.350	94 872
anti-homologous (control)	79/200	0.395	94 812

comparison	what it tests	Fisher exact p
homologous vs. random crossover	does homology beat random?	0.238
anti vs. homologous	**the control must be worse**	**0.408**
mutation-only vs. homologous	does the repair recover mutation?	**0.019**
mutation-only vs. anti		0.158

The repair fails on every clause of its own prediction. Homology restriction does not significantly beat random crossover. It does not recover mutation-only, which beats it significantly. And the negative control — built to be worse — is nominally *higher* and statistically indistinguishable from the treatment, which is precisely the outcome we had registered in advance as meaning the coefficient sign pattern does not carry the mechanism.

5.3 Ruling out the obvious confound

Both homology arms fall back to mutation when no admissible site exists, so a sceptic's first move is to attribute their edge over random crossover to extra mutation. The counts exclude it. Fallback rates are 2.35% (homologous) and 3.91% (anti-homologous) of recombination attempts; since recombination is attempted on half of iterations, the effective mutation fractions are 0.5117 and 0.5196 against random crossover's 0.5000. A difference under two percentage points in the operator mix cannot account for a 10.5 percentage point spread in solve rate. The ordering of the three crossover arms is therefore *not* explained by the fallback — and we leave it unexplained rather than supply a story, since no adjacent pair among them is significant.

5.4 What survives

What survives is the negative result, now from five independent directions: behavioural profiles are not heritable sequences; the address-indexed ablation contribution is not a site-wise heritable character; a mutation-only arm carries quartet certificates at the same rate as a crossover arm; random subtree crossover significantly lowers the solve rate; and the homology repair that the exact mechanism recommends does not recover the loss, with its own control refuting the mechanism. In this representation, the only operator that reliably works is mutation.

We note what this does *not* license. It does not show that no contribution-preserving recombination can work — only that this one, defined by the sign pattern of the context coefficient, does not, and that its failure cannot be rescued by the mechanism we proposed. A coefficient-matching scheme that *rescales* the donor to preserve $B \cdot v$ rather than merely matching the sign of B remains untested, and is the obvious next candidate. We report the failure now rather than after finding a variant that works, because the prediction and its falsifier were fixed before the run.

6 Preserving the contribution exactly

Edition 1 closed by naming the successor it had not tested: matching sign(B) failed, so the obvious candidate was to *rescale* the donor and preserve $B \cdot v$ itself. Edition 2 tests it.

6.1 The operator

A site is admissible only when $B_{\text{donor}} = k\, B_{\text{site}}$ for a *single* rational k at every sample point — one scalar cannot correct a point-varying ratio — and the donor is then inserted as $k \cdot D$, restoring $B_{\text{donor}}\, v$ exactly. The case $k = 1$ needs no wrapper and is common, since $B = 1$ at every address reached through only $+$ and the left of $-$, which is also why an earlier fidelity measurement found 1.000 in the $\{+, -\}$ arm.

The invariant is measured, not asserted: 1614 of 1614 contribution comparisons are exactly equal before and after the graft, in exact rational arithmetic, with admissible sites found in 269 of 400 random donor/site pairs, so the condition is neither vacuous nor trivial.

6.2 A sharper control than edition 1's

`same_sites_unscaled` selects the *same* constant-ratio sites and omits the scaling. This separates the two things the operator does — choosing a site, and correcting the contribution once there — which edition 1's design could not.

6.3 Result

arm	solved	rate	median evaluations
mutation-only	94/200	0.470	1286.5
crossover (random)	58/200	0.290	1269.0
homologous (edition 1)	70/200	0.350	1386.5
anti-homologous (edition 1)	79/200	0.395	1712
contribution-preserving	73/200	0.365	1011
same sites, unscaled (control)	52/200	0.260	1358.5

comparison	role	p	Bonferroni
preserving vs. unscaled	isolating test	0.031	no
preserving vs. mutation-only	does it rescue crossover?	0.042	no
preserving vs. crossover	**registered falsifier**	**0.136**	no
unscaled vs. crossover	sanity	0.576	no
preserving vs. homologous	exact vs. sign	0.835	no

6.4 The falsifier fired

We registered, before running: *if contribution-preserving does not significantly exceed random crossover, then exact contribution preservation is also not the missing ingredient, and the contribution-transfer story is refuted as an explanation rather than merely in its sign-matching form.* It does not ($p = 0.136$). The hypothesis fails the bar we set for it.

Five comparisons were run, so a family-wise rate of 0.05 requires $\alpha = 0.01$, and *nothing survives correction* — including the two raw-significant results.

6.5 What the data do support, at their real strength

The isolating comparison is the interesting one. Holding site selection fixed, turning the scaling on beats turning it off at $p = 0.031$, and the sanity check holds: the unscaled arm is indistinguishable from random crossover ($p = 0.576$), so site selection alone buys nothing and any effect lives in the scaling. That is the first positive mechanistic signal in this programme.

It is also not enough, and we decline to promote it. It fails correction for the five comparisons, and it is the comparison against the *weakest* arm; elevating it to the headline would be choosing the test after seeing the data, which is the defect this programme keeps finding in its own work. Three further things are therefore *not* claimed: that the scaling works, since the registered bar was exceeding random crossover and it was not met; that the operator is faster conditional on solving, though its median of 1011 evaluations against mutation-only's 1286.5 is suggestive at $p = 0.057$; and that exact preservation beats sign-matching, since $p = 0.835$ — the stronger invariant, verified exact at every one of 1614 comparisons, bought nothing measurable over matching a sign.

A 6-seed smoke test had shown 3/6 at a median of 475 evaluations. It was reported as a reason to run at 200 seeds rather than as a result, and at 200 the arm fails its bar. That is the third small-sample estimate in this work not to replicate.

6.6 What this leaves

Six independent results agree that subtree recombination does not help in this representation, and *both* mechanism-targeted repairs — matching $\text{sign}(B)$, and preserving $B \cdot v$ exactly — fail their pre-registered bars. The contribution-transfer account of why crossover fails survives only as a hint from one uncorrected sub-comparison, which is not enough to call it the cause. We state that rather than the alternative, which would be to keep proposing variants until one clears a bar set afterwards.

7 Reproduction

Every numeric claim above is produced by a script in the accompanying repository. The search, calibration, controls and gate are a single file; the verifier changes and their tests are in the main project. The artifacts carry the measured values, the pre-registered criteria, and the limits of each claim, including the ones that qualify it.

8 Related work and honest placement

This is a negative result about an operator, measured in one setting, with a mechanism and a failed or successful repair reported as it came out. It does not overturn the genetic-algorithm literature, where crossover's value has been both defended and attacked for decades and where representation is known to matter enormously. Its contribution is methodological as much as empirical: the measurement was affordable only after we found that our own verifier had been

reporting a limit of its own making, and the discipline that caught that — measuring a limit one layer below the code that reports it — is the part most likely to transfer.